

LAB1 REPORT

Name: Mourya Nandan

Roll No: CS23B006

What we want to do:

- Do initial lab setup then .
- Write, compile, and

run a HelloWorld.S program which prints "Hello < Your Roll No >! Welcome to "CS3500" inside xv6 qemu. Submit the following:

- The assembly code.
- The screenshots showing this executing on your system.

Initial Lab setup:

- Install the docker and run the commands in terminal

```
1)docker pull svkv/riscv-tools:v1.0
```

```
2)docker run -it svkv/riscv-tools:v1.0
```

- Then run cd in WSL then run the command

```
git clone https://github.com/mit-pdos/xv6-riscv
```

it downloads the github repo into our system location

- Write the file of your code with <name>.s in users folder of xv6-riscv/user then write
write you code whatever you wants and add \$U/_<name>/ in UPROGS.
- Then run the command to connect your terminal to the docker container

```
docker run -it -v <path to xv6-riscv in your host system>/xv6-riscv:/home/os-iitm/xv6-riscv svkv/riscv-tools:v1.0
```

- Then go to the xv6-riscv folder by running

```
cd xv6-riscv
```

then make clean that cleans the emulator

then run make qemu(its an emulator) it boots the kernel

then we want to run our .s file it gives the output which we written in code.

Explaining the code:

CODE:

```
1  .section .data
2  msg:
3      .asciz "Hello CS23B006! Welcome to CS3500\n"
4
5  .section .text
6  .globl main
7  main:
8      li a0, 1
9      la a1, msg
10     li a2, 34
11     li a7, 16
12     ecall
13
14     # exit(0)
15     li a0, 0
16     li a7, 2
17     ecall
```

1ST BLOCK:

```
1  .section .data
2  msg:
3      .asciz "Hello CS23B006! Welcome to CS3500\n"
```

- **.section .data** – specifies the following data will be placed in data segment of memory.
- **msg** – representing the starting address of that string in memory.
- **.asciz** – stores the string in memory of asciz format.

2nd BLOCK:

```
5  .section .text
6  .globl main
```

- **.section .text** – Switches to the code segment where the programs instructions are stored.
- **.globl main** – makes the main label visible to the linker file and treat main as entry point(we can use global as well).

3RD BLOCK:

```

7   main:
8       li a0, 1
9       la a1, msg
10      li a2, 34
11      li a7, 16
12      ecall

```

What actually going here is:

- When ecall runs CPU switches to kernel mode and check the value of a7 that is system call number.
- so when we load 16 into a7 that means it is calling system write(write function).
- so that system write takes 3 arguments as a0,a1,a2.
- a0 – specifies number if 0 stdin, if 1 stdout like that.
- a1 – specifies the address of the message what we want to print.
- a2 – specifies the length of the message we want to print, if we given large numbers it might print some garbage values.

4TH BLOCK:

```

14      # exit(0)
15      li a0, 0
16      li a7, 2
17      ecall

```

What actually going here is:

- When ecall runs CPU switches to kernel mode and check the value of a7 that is system call number.
- so we are loading 2 in a7 that means system exit(exit function).
- that function takes 1 argument a0.
- why we load 0 means it is the success and safe exit.

How to run:

After completing the code if we go to terminal and after connecting to the os-iitm container then if we run `make clean` then `make qemu` we get the output like this by a `$` symbol after entering our program name it will run successfully.

make clean – update the changed code. if you change any code after knowing the error so by running the `make clean` it update the change.

`make qemu` – an emulator which do xv6 kernel booting.

OUTPUT:

```
xv6 kernel is booting

hart 2 starting
hart 1 starting
init: starting sh
$ HelloWorld
Hello CS23B006! Welcome to CS3500
$ |
```